Infinite War?

The task is an optimization problem. The reference solution is a collection of heuristics.

The initial observation we need to make: If I have a card with value x and all cards in my opponent's deck are smaller than x, they have no way of taking card x from me.

As a corollary, whoever starts with card n wins.

Once we make these observations, we can have the intuition that largest card(s) in the loser's deck seem important. While the small cards will get swapped back and forth between decks multiple times, as soon as the largest card of the loser's deck goes away, we can be sure that it's never coming back: measurable progress towards the end of the game.

Digression: Proof of finiteness

Technically we don't need anything in this section to actually solve the task, so feel free to skip it if you are not interested.

Claim: For all initial configurations Danka can always end the game in a finite number of steps.

Proof #1 (non-constructive via graph theory):

Cases with $n \leq 2$ are obvious, so let's assume $n \geq 3$.

Consider the entire state space as a huge-but-finite directed graph: Each node is a specific pair of decks. Each node that is not the end of the game has outdegree 2, with the outgoing edges leading to the two states that can be reached by playing one round of the game (one produced by choosing option 0, the other by choosing option 1).

Let's use the color green for all states in which each deck has at least two cards. Each of these states has not just outdegree but also indegree equal to 2. This is because we have exactly two options which deck won the previous round, and each of those options leads to a unique previous state. (The bottom two cards of the winning deck were on tops, with the bigger one being on the winning deck.)

Next, let's use the color yellow for states in which one of the decks has just one card. These states have indegree 1.

Finally, let's color the terminal states red. These clearly have indegree 1 and outdegree 0.

Let $\mathcal{G}$ ("good") be the set of all good states from which we can reach a terminal state, and let $\mathcal{B}$ ("bad") be the set of all bad states from which this isn't possible. We want to show that $\mathcal{B}$ must be empty.

By contradiction. Assume $\mathcal{B}$ is non-empty. Each node in $\mathcal{B}$ has outdegree 2, as they are not terminal. There can be no edges from $\mathcal{B}$ to $\mathcal{G}$, therefore all $2|\mathcal{B}|$ outgoing edges lead back to $\mathcal{B}$. Each node in the whole graph has indegree at most 2. Thus, there is only one way in which the above is possible: each node in $\mathcal{B}$ has indegree exactly 2 and both incoming edges are from $\mathcal{B}$. This implies that:

- There are no edges from $\mathcal{G}$ to $\mathcal{B}$. (We won't need this.)
- All states in $\mathcal{B}$ must be green, because yellow ones don't have indegree 2.

Note that for each state in $\mathcal{B}$ each of the two incoming edges corresponds to another deck being the winning one in the previous step. Thus, if in the current state one deck has a cards and the other b, one of the previous states has deck sizes $(a-1, b+1)$ and the other has $(a+1, b-1)$.

We can now obtain a contradiction as follows:

- Pick any state in $\mathcal{B}$, and pick the deck in it that has more cards (either if equal).
- Repeatedly find and move to the previous state where that deck is one card bigger.
- Eventually the other deck must get down to just a single card, which is a contradiction because we just reached a yellow state after claiming that everything in $\mathcal{B}$ is green.

Proof #2 (semi-constructive but slow as a solution):

Lemma: For all k, if cards with numbers $\geq k$ are not all in the same deck, Danka can ensure that one of these cards eventually moves between decks.

Proof (by induction):

For $k = 1$ this is obvious: we just play the first round of the game and some card moves.

Induction step: Assume we already know that our statement is true for $k-1$, we are going to show it for k.

By contradiction. Suppose we cannot do it, which means that Danka can never make two of the large cards meet.

Note that our decks satisfy the requirement for $k-1$. That is, if we know that cards $k \dots n$ are not all in the same deck, we also know that cards $(k-1) \dots n$ are not in the same deck. Hence, the induction hypothesis applies to our two decks: Danka can eventually make one of these cards change decks. But as it's not possible for cards with numbers $\geq k$, the card that will change decks must be precisely card $k-1$.

Once that happens, the two new decks *still* satisfy the requirements for the induction hypothesis to apply, and thus Danka can make card $k-1$ change decks again. (And again, and again.)

Let A be the deck that initially contained the card $k-1$ and let B be the other deck. When $k-1$ first moves from A to B, it must be because it lost to some bigger card ℓ in that deck.

Now let's continue playing the game until the moment when $k-1$ moves back from B to A. At this moment, card $k-1$ met (and lost to) another large card m from deck A.

Remember that when we started this proof by contradiction, we made the assumption that Danka can never make two of the large cards meet. Now we are about to get the contradiction.

In all of the rounds since $k-1$ moved to B, cards $k-1$ and ℓ, if they were played, only encountered cards smaller than $k-1$ and won each such round. Now consider what would happen if we went back in time to the moment when $k-1$ lost to ℓ and *inserted these two cards to the bottom of B in the opposite order*. Clearly, all rounds would play out exactly the same (only with $k-1$ and ℓ appearing instead of each other),

and thus we would again reach the round in which originally $k-1$ met the bigger card m from deck A. However, this time the two large cards ℓ and m meet, which is our contradiction.

Corollary: Whenever card k is in one deck and all cards $> k$ are in the other, Danka has a way of moving k to the other deck.

Corollary of that Corollary: Danka can proceed from $n-1$ down to 1, always making sure that the current card is in the same deck as card n.

That's it for the proofs, now we can go back to heuristics for solving the game quickly.

Outer framework: worst-first refinement

As our score depends on the worst game played, it makes sense to keep track of the best known way of playing for each test case and always trying to optimize the worst one among them. This is easily implemented by keeping all test cases in a priority queue sorted according to the length of the best known solution, longest first.

Easy early solutions

A purely random solution is too inefficient, on our 1000 test cases its median is somewhere around 160k steps and worst case will be well over 1M steps. This is still firmly in the zero points territory.

However, simply adding the outer framework described above helps significantly: if we prioritize always trying another random playthrough for the game that currently takes the longest, we will quite quickly get rid of all the really long games. In our experiments, already in 10 minutes (single core, regular laptop) this solution easily got below $x = 60\,000$ and scored about 35 points. Running it longer (and possibly on multiple cores) can squeeze out a bit more — it's probably possible to get to $x = 45\,000$ and score about 43 points.

In order to score better, we need a better heuristic. One that's still easy and quick to implement: instead of randomly playing an entire game at once, let's divide each game into stages. In each stage we look at the largest card ℓ in the eventually-losing deck. We play random games, but each time only until the moment when ℓ loses to a bigger card and moves to the eventually-winning deck. Once we have many such game segments, we pick the shortest one and continue from there.

Our implementation of this simple greedy strategy, when executed for 10 minutes, got below $x = 24\,000$, scoring around 63 points.

Directly pairing cards

Sometimes, when we choose between options 0 and 1, we can directly tell something about what effect this choice will have in the future.

Consider the situation where the decks A and B had sizes (a, b) with $a + 2 \leq b$, and then in the next round the first deck won.

The cards added to the bottom of A will be cards a and $a+1$ in this deck. They will eventually get played against cards at positions a and $a+1$ in deck B. As B is big enough, both of these cards are already there

and we know what they are. Hence, whenever we are in this situation, we can immediately tell what two new pairings we are creating in the future.

(As a technical detail, note that for even n the conditions $a + 2 \leq b$ and $a < b$ are equivalent.)

Our best-performing heuristic

Heuristic for a single turn: Let L be the eventually-losing player. If the turn winner's deck is not the shorter of the two, we play arbitrarily (e.g., always option 0, or randomly). Otherwise, we can apply the argument from the previous section, and we'll pick the better of the two options according to the following criteria:

- We minimize the number of turns L is going to win.
- We minimize the largest card L would collect in the set of cards won (including those that come from L's deck).
- We minimize the sum of cards L would collect.

(The later criteria are tiebreakers for the previous ones. In all cases we are looking at the two future turns that will involve the two cards we are just placing on the bottom of the shorter deck.)

Base play: From any game state we simulate the game forward step by step, applying the above heuristic whenever we can. Playouts that exceed the length of the current best solution are terminated as unsuccessful.

In order to get some additional perturbation between runs, we may occasionally (with some small probability) make random moves instead.

Improved play: Instead of just blindly applying the heuristic in each step, we can add a lookahead. A one-step lookahead would work as follows:

- Try out both options for the next move.
- For each of those, run the base play algorithm.
- Commit to the option that produced the shorter game.
- Update the best answer found whenever it happens during the process.

The same can be safely done for a bigger lookahead, such as 6 steps (i.e., trying $2^6 = 64$ options).

Level 2 improved play: Instead of evaluating the two options using base play, we can evaluate them using improved play in each step. Note that a single call to the improved play procedure is still linear in the game length, and thus a single call to this new procedure will be quadratic (not exponential).

Best results found

Within a reasonable amount of time, our best solution managed to achieve $x = 9\,472$, which corresponds to the score of 92.59 points.